



Práctica

Presentación

Por fin llegamos al final del curso. Esta práctica servirá para sintetizar todos los conocimientos del curso y ampliarlos con el diseño de circuitos más complejos. En esta práctica se os pedirá el diseño de un circuito a partir de un grafo de estados y el diseño de una máquina de estados en concreto. Para la primera parte, tendréis que aplicar los conocimientos que habéis adquirido en este curso como, por ejemplo, los mapas de Karnaugh o los sistemas de representación. Por lo tanto, se recomienda que le deis un vistazo.

Competencias

Conocer la organización general de un computador como circuito digital y conocer los rasgos distintivos de la arquitectura de Von Neumann.

Objetivos

- Conocer varios modelos de las máquinas de estados y de las arquitecturas de controlador con camino de datos.
- Haber adquirido una experiencia básica en la elección del modelo de máquina de estados más adecuado para la resolución de un problema concreto.
- Ser capaz de diseñar circuitos secuenciales a partir de grafos de transiciones de estados.

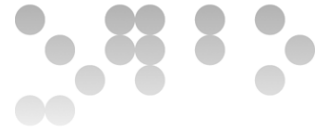
Recursos

Los recursos que se recomienda usar para esta práctica son los siguientes:

- **Básicos:** El módulo 5 de los materiales.
- **Complementarios:** VerilCIRC, VerilCHART y el Wiki de la asignatura.

Criterios de valoración

- Razonad la respuesta en todos los ejercicios. Las respuestas no justificadas no recibirán puntuación.
- La valoración está indicada en cada uno de los subapartados.



Formato y fecha de entrega

- Para dudas y aclaraciones sobre el enunciado, dirigiós al consultor responsable de vuestra aula.
- Hay que entregar la solución en un documento PDF usando una de las plantillas entregadas junto con este enunciado.
- Se debe entregar a través de la aplicación de **Entrega y registro de EC** del apartado Evaluación de vuestra aula.
- La fecha límite de entrega es el **18 de mayo**, a las 24 horas.

Solución de la práctica propuesta

PRIMERA PARTE [65%]

Dada la máquina de estados siguiente:

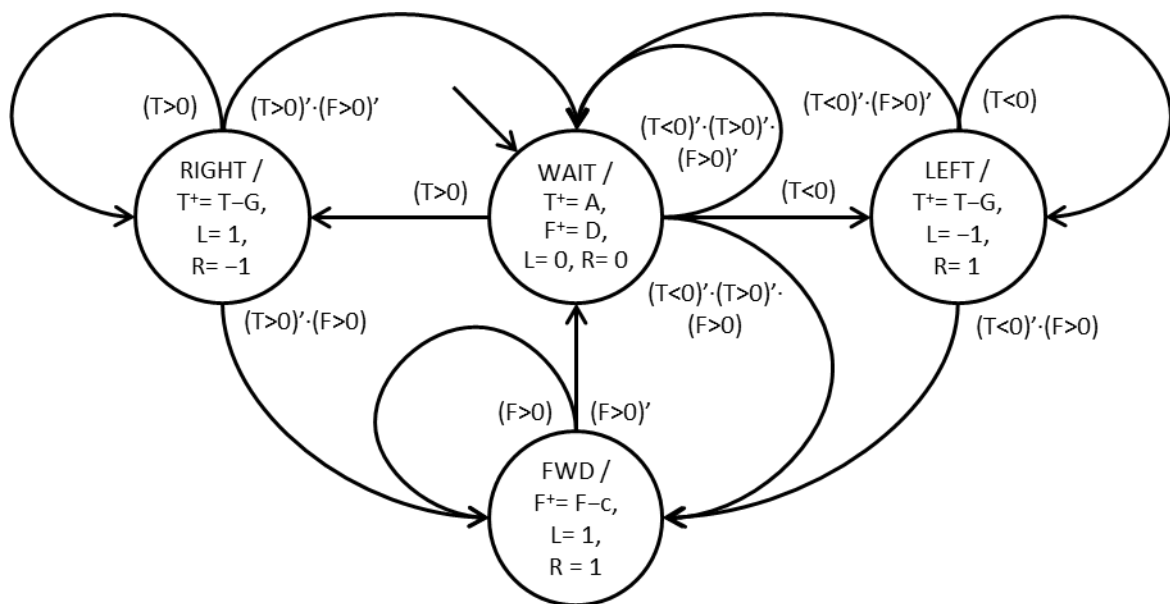
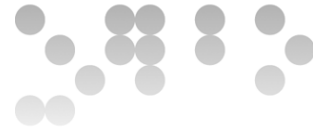


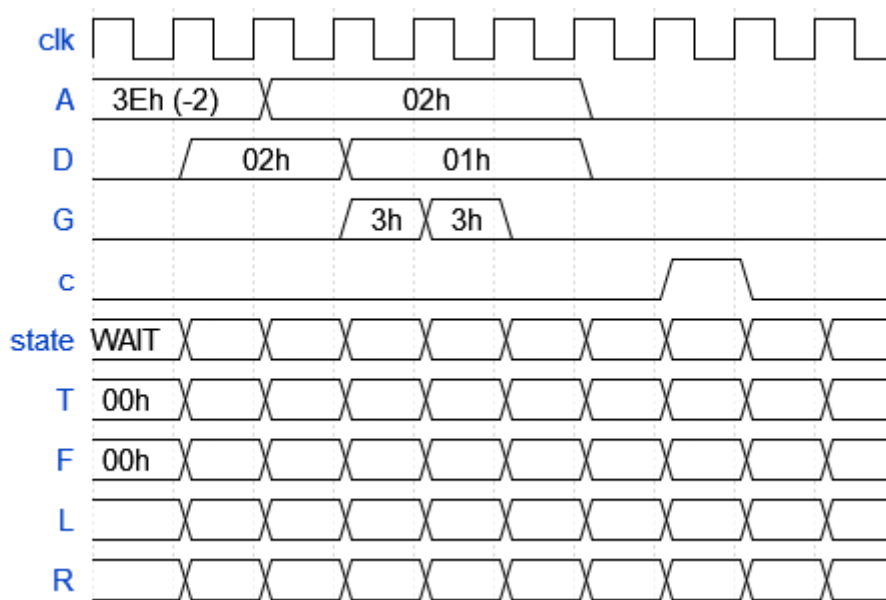
Fig. 1. EFSM de un controlador de un robot móvil.

Sabiendo que la entrada A es una señal de 6 bits en complemento a 2, que la entrada D es también de 6 bits, pero sin signo (números naturales); que T y F tienen el mismo formato que A y D , y que la entrada G es de 2 bits en complemento a 2, igual que las señales de salida L y R , se pide que:



- a) [15%] A partir del grafo de la EFSM de la fig. 1 completad, en el cronograma siguiente, la evolución del estado (*state*) y de las señales *T*, *F*, *L* y *R* en cada ciclo de reloj.

Nota: Tenéis el ejercicio disponible en VerilChart, donde el estado es *E* y los valores de las señales numéricas se representan en hexadecimal.



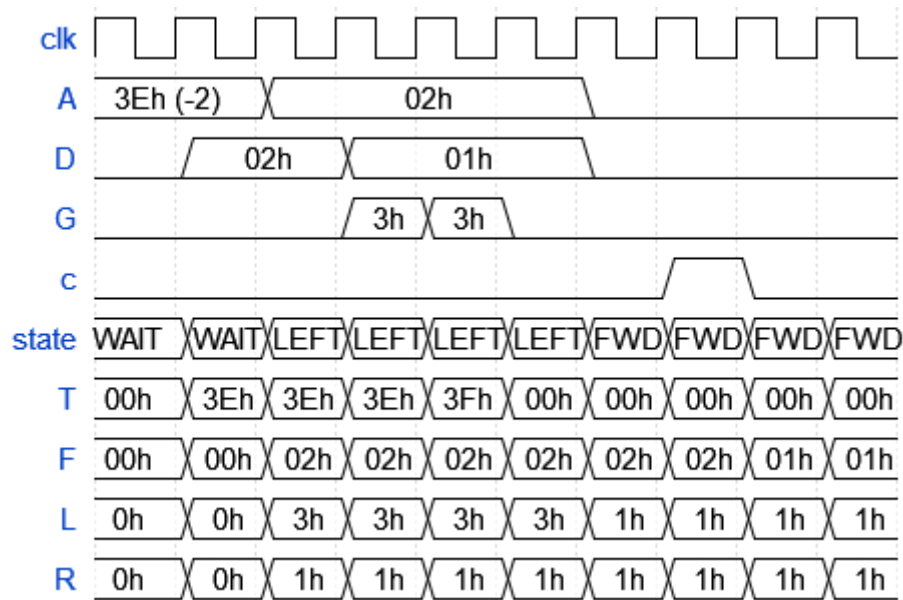
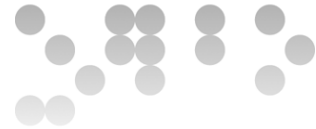
A partir de las condiciones iniciales que hay en el cronograma ($E=WAIT$, $A=-2$, $D=0$, $G=0$, $c=0$, $T=0$ y $F=0$) se puede determinar el estado siguiente y los nuevos valores para T y F .

El valor de las señales L y R depende de forma exclusiva y combinacional del estado en que se encuentra la máquina en cada momento. Como la máquina está en el estado $WAIT$, los valores son $L=0$ y $R=0$.

El siguiente valor de las variables T y F será el resultado del cálculo que se indique en el estado actual, es decir, en el estado $WAIT$, en este caso. En el nuevo ciclo de reloj, T tomará el valor que tenga la entrada A y F el valor que tenga la entrada D . Es decir, -2 y 0 , respectivamente.

Determinar el valor del estado siguiente implica determinar cuál de las transiciones se activa. En este caso, T y F tienen el valor 0 , lo cual hace que el estado siguiente vuelva a ser $WAIT$, dado que se cumple la condición $(T < 0)' \cdot (T > 0)' \cdot (F < 0)'$.

Siguiendo este procedimiento, se pueden determinar los valores en cada ciclo de reloj, a partir de los valores del ciclo anterior y se puede ir repitiendo el procedimiento hasta completar el cronograma, tal como se muestra en la figura siguiente.

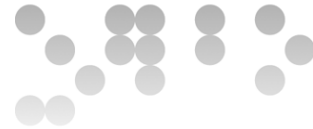


- b) [15%] Obtened las tablas de transiciones y de salidas de la EFSM de la fig. 1, así como las codificaciones binarias de estados y salidas correspondientes.

En el grafo tenemos las señales de entrada ($T < 0$), ($T > 0$) y ($F > 0$) que condicionan las transiciones. También tenemos las entradas A , D , G y c , pero sólo se utilizan en el cálculo del valor futuro de las variables T y F , de las cuales tenemos que determinar los valores que toman, como también los hemos de determinar para las salidas L y R .

Las tablas de transiciones y salidas que se obtienen a partir del grafo son:

Estado actual	Entrada	Estado siguiente	Estado	Salidas
WAIT	$(T < 0)$	LEFT	WAIT	$L=0, R=0, T^+ = A, F^+ = D$
WAIT	$(T > 0)$	RIGHT	RIGHT	$L = 1, R = -1, T^+ = T - G, F^+ = F$
WAIT	$(T < 0)' \cdot (T > 0)' \cdot (F < 0)'$	WAIT	LEFT	$L = -1, R = 1, T^+ = T - G, F^+ = F$
WAIT	$(T < 0)' \cdot (T > 0)' \cdot (F < 0)$	FWD	FWD	$L=1, R=1, T^+ = T, F^+ = F - c$
RIGHT	$(T > 0)$	RIGHT		
RIGHT	$(T > 0)' \cdot (F > 0)'$	WAIT		
RIGHT	$(T > 0)' \cdot (F > 0)$	FWD		
LEFT	$(T < 0)$	LEFT		
LEFT	$(T < 0)' \cdot (F > 0)'$	WAIT		
LEFT	$(T < 0)' \cdot (F > 0)$	FWD		
FWD	$(F > 0)'$	WAIT		
FWD	$(F > 0)$	FWD		



Para la codificación binaria, hay que asignar en cada estado un código binario individual. En este caso, se sigue el criterio más habitual, que es el de la numeración binaria, desde el 0 para el estado inicial hasta el número de estados que haya menos 1: 00 para WAIT, 01 para RIGHT, 10 para LEFT y 11 para FWD.

Para elegir el valor siguiente que se asigna a las variables T y F hacen falta dos selectores de dos bits cada uno para poder elegir entre tres valores posibles para cada una de ellas. Alternativamente, se puede usar el código de estado como selector:

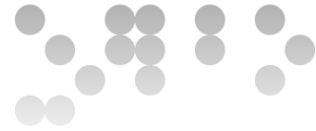
Operaciones	Estado	Efecto sobre la variable
$T^+ = A, F^+ = D$	00	Asignar Ah a T y Dh a F
$T^+ = T - G, F^+ = F$	01	Asignar a T el resultado del cálculo $T - G$ y mantener el valor de F
$T^+ = T - G, F^+ = F$	10	Asignar a T el resultado del cálculo $T - G$ y mantener el valor de F
$T^+ = T, F^+ = F - c$	11	Mantener el valor de T y asignar a F el resultado del cálculo $F - c$

El valor de las salidas L y R se determina directamente a partir del estado, bien combinacionalmente, bien usando el código del estado como selector.

Finalmente, las tablas de transiciones y de salidas con la codificación binaria correspondiente son:

Estado actual		Entrada			Estado*
Símbolo	q_1q_0	($T < 0$)	($T > 0$)	($F > 0$)	$q_1^+q_0^+$
WAIT	00	0	0	0	00
WAIT	00	0	0	1	11
WAIT	00	1	x	x	10
WAIT	00	x	1	x	01
RIGHT	01	x	1	x	01
RIGHT	01	x	0	0	00
RIGHT	01	x	0	1	11
LEFT	10	1	x	x	10
LEFT	10	0	x	0	00
LEFT	10	0	x	1	11
FWD	11	x	x	0	00
FWD	11	x	x	1	11

Estado		Selector
Símbolo	q_1q_0	q_1q_0
WAIT	00	00
RIGHT	01	01
LEFT	10	10
FWD	11	11



Notad que, en la columna “Selector” se ha repetido el contenido del estado. Para seleccionar los nuevos valores de T y F se usa el código del estado. Si se usan señales diferentes a estas, también es correcto. (Tened presente, además, que, según la codificación elegida para los estados y para el selector, el resultado de la tabla anterior puede variar.)

El controlador de un caudalímetro (Fig. 2, izquierda) recibe dos señales de entrada que indican si ha pasado un litro de agua a través del tubo del sensor (u) y si ha pasado un minuto (m). El caudal (F) se actualiza cada minuto con el número de litros que han fluido a través del sensor durante el último minuto y la señal de salida v indica si hay exceso de caudal, es decir, por encima de 20 litros por minuto o no. Además, por limitaciones físicas, no pueden pasar más de 30 litros por minuto por el tubo.

El comportamiento de este controlador se representa en el diagrama de estados de la figura siguiente.

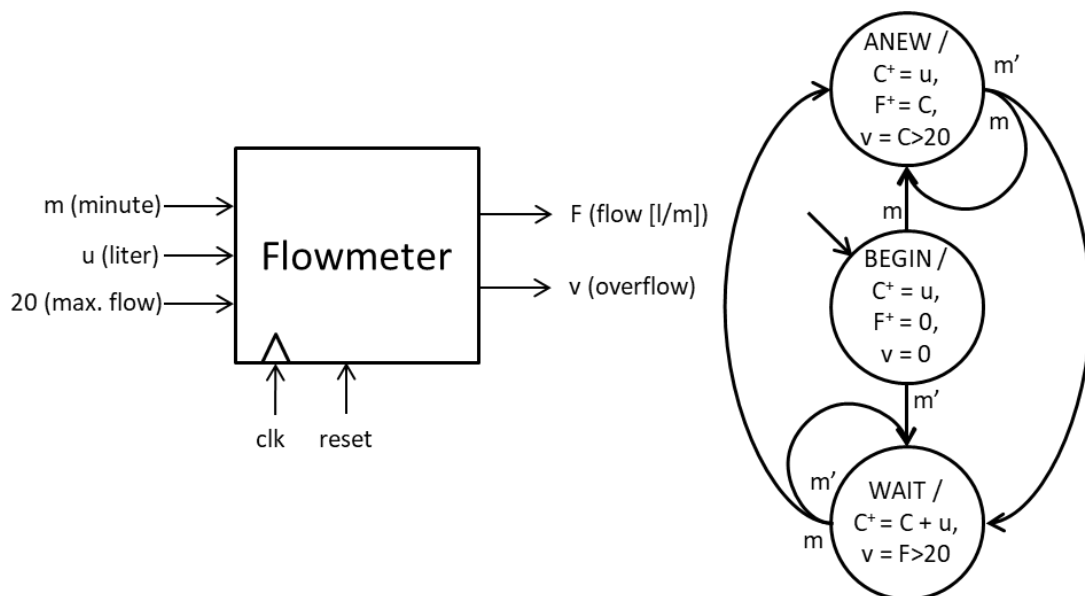
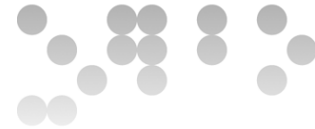


Fig. 2. Esquema y EFSM del caudalímetro.

A la vista de la EFSM correspondiente, se pide que:

- c) [15%] A partir de las tablas de transiciones y de salidas del grafo de la fig. 2 que se dan a continuación, implementad la unidad de control usando ROM y los registros, los bloques combinacionales y las puertas lógicas que creáis convenientes. Especificad y justificad las dimensiones de la/las ROM que uséis e indicad el contenido en binario, indicando a qué corresponde cada bit, y en hexadecimal.



Estado actual	Entrada	Estado*
	m	
BEGIN	0	WAIT
BEGIN	1	ANEW
WAIT	0	WAIT
WAIT	1	ANEW
ANEW	0	WAIT
ANEW	1	ANEW

Estado		Salidas		
Símbolo	q_1q_0	s_C	s_F	s_v
BEGIN	00	0	00	00
WAIT	01	1	01	01
ANEW	10	0	10	10

Nota: Tenéis el ejercicio disponible en VerilCIRC y, si queréis comprobar previamente el correcto funcionamiento de la ROM, la tenéis disponible en VerilChart. En este entorno, el nombre de la señal de estado es *E*.

Para implementar la unidad de control con una ROM hace falta, primero, determinar el tamaño y, después, calcular el contenido a partir de la tabla de transiciones correspondiente.

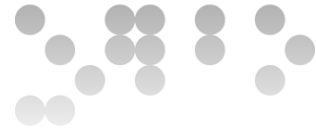
El bus de direcciones de la ROM debe ser de 3 bits, puesto que la dirección se forma con los bits que codifican el estado (son 2, en este caso) y los de las señales de entrada (1 más). Por lo tanto, la ROM tiene que disponer de un total de $2^3 = 8$ posiciones de memoria.

Cada posición guarda el código del estado siguiente y el valor, en el estado actual, de cada una de las señales de salida. Por lo tanto, hacen falta 2 bits para el estado y harían falta 5 bits para las salidas (*s_C*, *s_F*, y *s_v*), lo que haría un total de 7 bits.

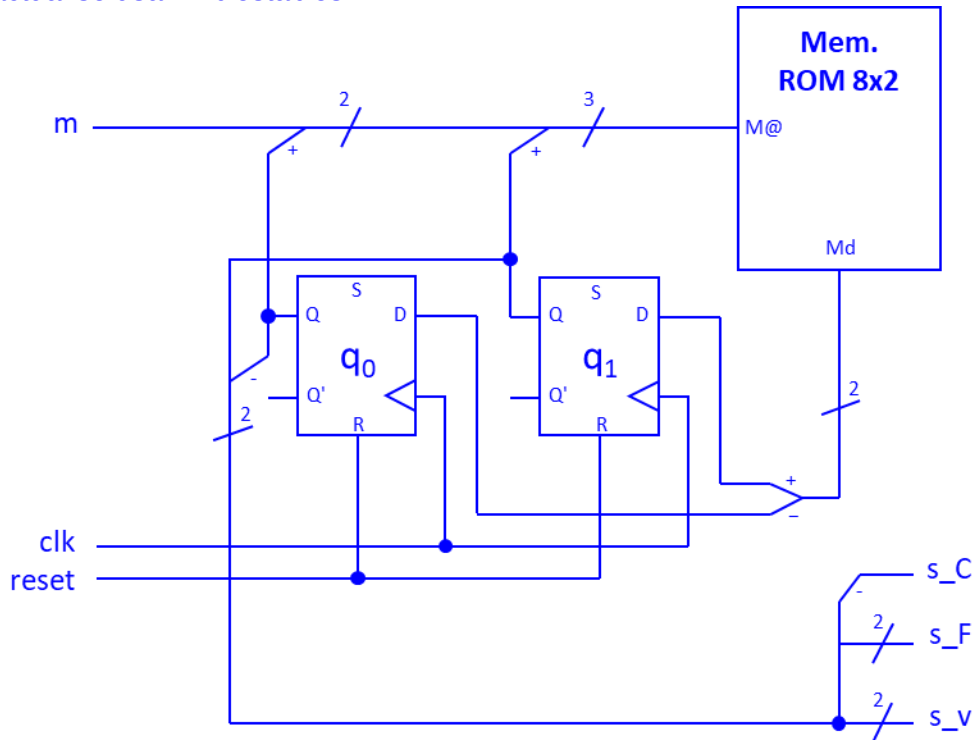
De todos modos, se puede observar que los selectores *s_F* y *s_v* coinciden con el valor del estado actual y, por lo tanto, no hay que almacenarlos en la ROM. Lo mismo ocurre con *s_C*, que es el bit menos significativo del código del estado actual, q_0 .

Con todo, la ROM es sólo para el cálculo del estado siguiente, con una dimensión de 8 palabras de 2 bits. El contenido de la ROM se determina a partir de la tabla de transiciones:

Posición de memoria		Contenido	Codificación hexadecimal
q_1q_0	m	$q_1^*q_0^*$	
00	0	01	1 h
00	1	10	2 h
01	0	01	1 h
01	1	10	2 h
10	0	01	1 h
10	1	10	2 h
11	0	00	0h
11	1	00	0h



La figura siguiente muestra la implementación con la ROM correspondiente. La entrada $M@$ de la ROM se configura con el valor del estado actual (los 2 bits de más peso) y el valor de la entrada (el bit de menos peso). Para guardar el estado actual se usan 2 biestables.



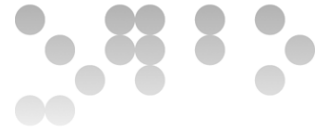
Hay que tener en cuenta que las señales de selección se obtienen directamente del estado actual, es decir, de las salidas de los biestables q_0 y q_1 y no de la memoria ROM.

- d) [20%] Implementad el circuito completo de la EFSM de la fig. 2: Construid el camino de datos usando las puertas lógicas y los bloques necesarios e incorporad la unidad de control obtenida en el apartado anterior. Indicad y razonad, para cada bus, su dimensión en bits.

Nota: Tenéis el ejercicio disponible en VerilCIRC. En el circuito, la señal C también es de salida para poder comprobar más fácilmente el funcionamiento. Tened en cuenta que VerilCIRC no puede verificar subcircuitos diseñados por el usuario. Por lo tanto, tendréis que copiar en vuestra solución el circuito de la unidad de control que hayáis diseñado en el apartado anterior.

Por un lado, el camino de datos debe ocuparse de generar las señales necesarias para la unidad de control. En este caso, sin embargo, sólo se usa la señal de entrada m y no hay que generar ninguna otra señal. De otro, el camino de datos también debe actualizar y almacenar el valor de las variables que se usen y generar los resultados de las operaciones que sean necesarias. Aquí se usan las variables C y F .

Sabemos que no es posible tener un caudal superior a 30 litros por minuto. Por lo tanto, la variable C podrá llegar como máximo a este valor. Esto determina que 5



SEGUNDA PARTE [35%]

Hay que diseñar el modelo del controlador de un paso de peatones regulado por un semáforo que se pone en verde durante 15 segundos después de que se haya pulsado el botón de petición q , con una espera de D segundos.

Esta espera de D segundos se introduce para evitar interrumpir el tráfico de vehículos demasiado a menudo. La primera vez, será de 5 segundos, después de 15, 25 y 35 segundos, respectivamente. Después de una espera de 35 segundos, la siguiente vez, volverá a ser de 5 segundos.

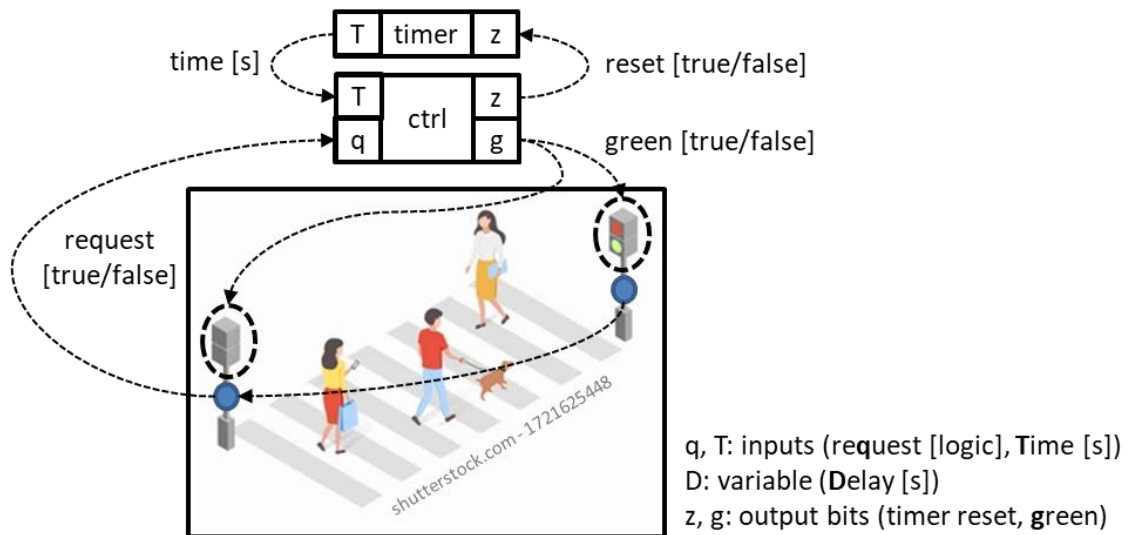


Fig. 3. Esquema de bloques del controlador de un semáforo de peatones.

Para tener en cuenta el tiempo, se dispone de un temporizador que proporciona el tiempo T en segundos que han transcurrido desde la última inicialización con $z=1$. (Si se mantiene z a 1, T se mantiene a 0.)

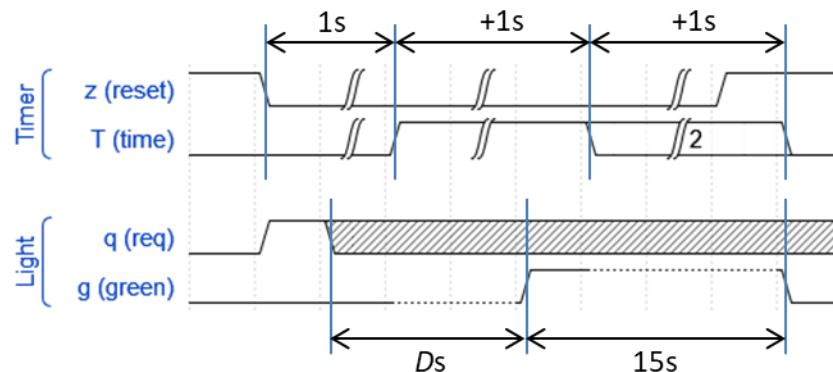
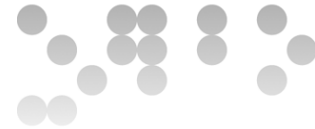


Fig. 4. Cronograma de funcionamiento del temporizador (parte superior) y de la puesta en verde retardada del semáforo (parte inferior).

Diseñad la ASM del controlador del semáforo de peatones que se ha descrito.



El controlador debe ocuparse de generar la señal g , para mantener encendida la luz verde ($g=1$) o la roja ($g=0$) de los semáforos, y la señal z para inicializar el temporizador.

El temporizador debe ponerse en marcha después de pulsar el botón ($q=1$) para contar los segundos de espera transcurridos. Por tanto, hay que monitorizar la señal q . Cuando se pone a 1, se inicia la espera correspondiente de 5 segundos la primera vez, de 15, 25 y 35 las siguientes veces sucesivas. Por lo tanto, tenemos un valor de espera inicial de 5 segundos que guardaremos en una variable y que se irá incrementando en 10 con cada nueva petición hasta un máximo de 35. Después de llegar al valor máximo de 35 se vuelve a las condiciones iniciales, donde la espera es de 5 segundos.

Una vez transcurrido el tiempo de espera, que podemos ir comprobando a través de la salida T del temporizador, el semáforo se pone en verde ($g=1$) durante 15 segundos. A continuación, se pone de nuevo en rojo ($g=0$) y se vuelve a monitorizar la señal q una vez actualizado el valor de la espera.

La figura siguiente muestra la ASM correspondiente a este comportamiento:

